causal_lm.py

`src/moe_pruning/tasks/causal_lm.py · 109 lines · module moe_pruning.tasks.causal_lm`

What it does

The decoder-only objective used for OLMoE: calibration is prompt + response in one sequence; evaluation is next-token prediction on documents.

How it works

- The prompt uses the tokenizer's chat template if it has one, otherwise `User: ...` / `Assistant:` .
- All tokens feed the MoE statistics; only response tokens are labelled.
- Loss uses shifted logits and labels; since this is a genuine language-model loss, perplexity is reported too.
- Qualitative output: a 64-token prompt, the reference continuation and the model's greedy continuation.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class CausalExample`	One evaluation document as token ids (labels = input ids).
`class CausalLMTask`	Decoder-only objective with next-token evaluation.
`.tokenize_calibration(records, tokenizer, max_source_length, max_target_length)`	`human` is the input/prompt, `response` the teacher-forced target.
`.collate(items, pad_token_id)`	Right-padded batch with per-stack real-token masks.
`.eval_settings(cfg)`	Task-specific settings that determine the evaluation examples.
`.build_eval_example(example_id, text, tokenizer, cfg)`	One evaluation example (must expose `example_id`, `input_ids`, `labels`).
`.token_nll(logits, labels)`	Per-example (sum of token NLL, number of scored tokens).
`.predictions(logits, labels)`	Per-example argmax token at every scored position (same order as `token_nll`).
`.qualitative(model, tokenizer, examples, device, cfg)`	`[{"example_id", "inputs": {label: text}, "prediction"}]` from greedy generation.

Settings it reads

`MAX_SOURCE_LENGTH`

Connections

Uses: `moe_pruning.data.calibration`, `moe_pruning.data.collators`, `moe_pruning.tasks.base`

Used by: `src/moe_pruning/models/__init__.py`, `src/moe_pruning/models/olmoe/__init__.py`